

An 8-bit Deep Learning Accelerator for GAN Image Generation on GF180MCU

William Anthony, I Made Medika Surya W.M., John Ruben Saragi, Mahardhika Putra Adipratama,
Fairuz Aquilla Rahagi, Trio Adiono, and Infall Syafalni

Bandung Institute of Technology, Bandung, Indonesia

E-mail: {13223048, 13222021, 13222049, 13222081, 13222107}@mahasiswa.itb.ac.id,
tadiono@itb.ac.id, infall@ieee.org

Abstract—This paper presents an 8-bit deep learning accelerator for a small GAN image generator, taken through a fully open-source flow to a tapeout-ready chip on the open GF180MCU 180 nm process. The target network is a three-layer multi-layer-perceptron generator trained on MNIST, quantized from FP32 to signed 8-bit integers and mapped onto a structural 4×4 processing-element array with a native accumulation depth of $K = 256$. The A, B, and C operand buffers use eleven physical 256×8 SRAM macros instead of inferred register arrays. The accelerator is hardened with LibreLane as a single-supply 3.3 V macro with zero DRC, LVS, and antenna violations at a 40 ns clock across nine STA corners, and is then integrated behind a 4-wire serial interface onto a $2935 \times 2935 \mu\text{m}$ padring, which again signs off clean at chip level, including chip-level LVS over 71,668 instances. Functional correctness is verified against Python golden vectors at every level, from a single matrix multiplication up to a full 784-pixel image rendered bit-exactly through the routed post-layout netlist.

Index Terms—Deep learning accelerator, GF180MCU, SRAM macro, INT8 quantization, GAN, ASIC, LibreLane, open-source EDA.

I. INTRODUCTION

Generative Adversarial Networks can be used as a compact workload for testing neural inference on custom hardware. In this work, the generator is a small MNIST multi-layer perceptron that converts a 64-dimensional latent vector into a 28×28 image. The arithmetic is mostly dense matrix multiplication, so the workload can be mapped onto a small array of multiply-accumulate units.

An earlier version of the design used sequential behavioral loops and register memory arrays in Verilog. That version was convenient for checking the algorithm, but not suitable for ASIC implementation: buffers inferred as standard-cell registers make the design too large and difficult to route. The design was therefore restructured into a structural accelerator with a processing-element (PE) array, local SRAM-backed buffers, and a small controller, and carried through a two-stage physical flow: the core is hardened as a macro, and the macro is integrated onto a padring for a shared-shuttle slot.

The main contributions are: (1) an open INT8 GAN-generator accelerator on GF180MCU whose buffers use physical SRAM macros; (2) a verification stack reaching from unit vectors to a bit-exact full-image render on the routed post-layout netlist; (3) a single-supply 3.3 V implementation of the

full flow, including third-party library fixes and a reproducible antenna-closure method; and (4) chip-level integration behind a 4-wire serial interface with a clean sign-off.

II. RELATED WORK

Most neural network accelerators are built around arrays of multiply-and-accumulate units. A well-known large-scale example is the systolic Tensor Processing Unit [1], and a broad overview of deep learning accelerators for different platforms is given in [2]. To run a floating-point network on this kind of integer array, the weights and activations are usually quantized with a per-tensor scale, as described by Jacob et al. [6].

Among fabricated accelerators, the reference points span three orders of magnitude in scale. DianNao [3] places 496 16-bit fixed-point operators in 3.02 mm^2 of 65 nm and reaches 452 GOPS at 485 mW. Eyeriss [4] spends a 12.25 mm^2 65 nm core on 168 processing elements with a row-stationary dataflow that minimizes data movement, and measures 278 mW at 200 MHz. Closer to our own scale, Wang et al. [7] present a small reconfigurable PE-array accelerator for an underwater recognition network in 40 nm, at 54.61 GOPS and 96.35 mW. All three are closed-PDK designs on commercial nodes.

The comparison that matters most for the contribution claimed here is against silicon built the same way. OpenSpike [5] is a spiking-neural-network accelerator taped out in the open SkyWater 130 nm process using only open-source EDA tools and OpenRAM-generated memory macros, occupying 33.3 mm^2 at 40 MHz. Our design shares that open-flow premise but differs in workload and memory strategy: it is an INT8 dense-matrix engine backed by foundry-supplied hard SRAM macros rather than a spiking architecture with generated memory, and it is carried through padring integration to a complete chip. Section IX places all of these side by side. The workload is a pretrained MLP GAN [8], [9], and the RTL is based on the open DLA Engine project [10].

III. GAN ARCHITECTURE

The GAN generator architecture, illustrated in Fig. 1, uses a multi-layer perceptron (MLP) model that receives a latent vector of size 64 as the initial input. This input is then projected through two hidden layers, each containing 256 neurons, to

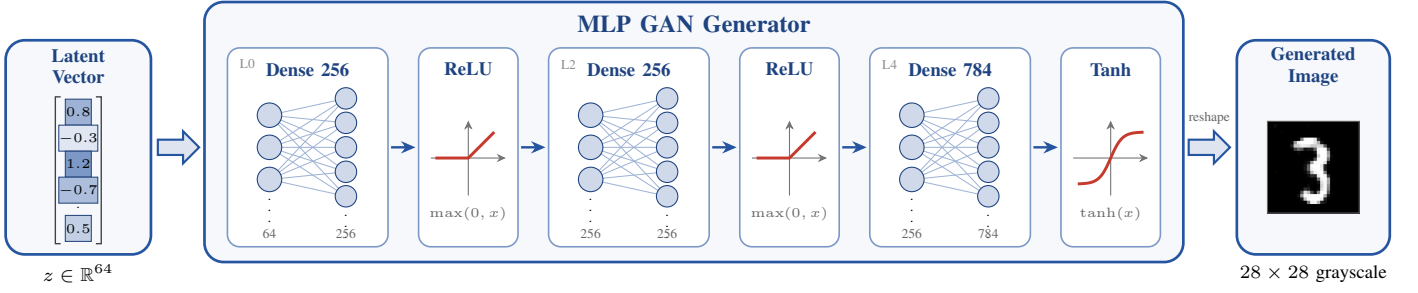


Fig. 1. MLP GAN generator: the 64-dimensional latent vector z is expanded through two 256-neuron ReLU hidden layers and a 784-neuron tanh output layer into a 28×28 grayscale image (layer tags match Table I).

form a richer feature representation. Each hidden layer is followed by a ReLU activation function, allowing the model to learn non-linear patterns from the data.

After that, the learned representation is passed to an output layer with 784 neurons, which corresponds to a 28×28 pixel image. In the final stage, a tanh activation function is used to limit the output values to a range between -1 and 1 , so that the output of the generator can be represented as a grayscale image. Through this process, the generator transforms a random vector into a synthetic image that resembles real data.

Table I lists the three dense layers and shows how many tiles each layer needs on the 4×4 array. Because the array produces four output neurons per tile, the layer with 784 outputs needs the most tiles.

TABLE I
GENERATOR LAYERS AND THEIR MAPPING TO THE 4×4 ARRAY.

Layer	Output $\times$ Input	Activation	Tiles	MAC ops
L0	256×64	ReLU	64	16,384
L2	256×256	ReLU	64	65,536
L4	784×256	Tanh	196	200,704
Total			324	282,624

IV. HARDWARE ARCHITECTURE

The DLA computes a tiled matrix multiplication $C = A \times B$. The array size is $N = 4$, and the native accumulation depth is $K = 256$, so one transaction accumulates a full 256-term dot product—the inner dimension of the widest generator layers. K is a temporal depth: it sets how many cycles the same 16 PEs accumulate, not the array size. An earlier RTL revision defaulted to $K = 4$ and relied on a harness override; since the physical flow synthesizes the bare RTL defaults, the hardened default was widened to $K = 256$ —a parameter-only change, as the operand SRAMs were already 256 deep. During one transaction, the controller clears the PE accumulators, enables the array for K cycles, and then stores the result block to the C buffer through a writeback path.

At the top level, shown in Fig. 2, the A buffer provides one row vector to the PE array, and the B buffer provides one column vector. The control unit generates the read index, clear

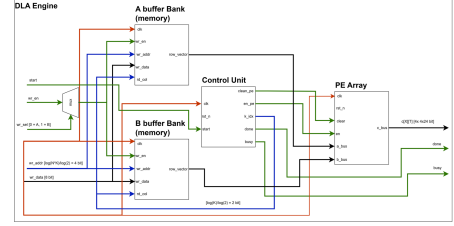


Fig. 2. DLA engine top level.

signal, enable signal, done signal, and busy signal. The output bus carries the accumulated C values.

A. Processing Element

Each processing element (Fig. 3) receives two signed 8-bit inputs. It multiplies them, sign-extends the 16-bit product to 24 bits, and adds it to a local accumulator. The accumulator is cleared at the start of each matrix multiplication. The output of each PE is one dot product.

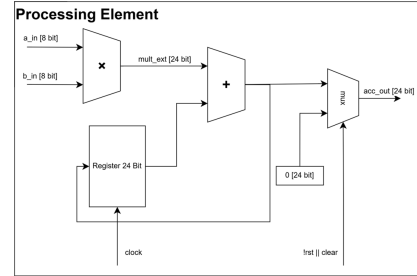


Fig. 3. Processing element datapath.

The main computation in one PE follows

$$acc_{out}[n] = acc_{out}[n-1] + a_{in}[n] b_{in}[n]. \quad (1)$$

After K cycles, the accumulator stores one output value of the matrix multiplication tile. A 24-bit accumulator is used so that the sum of 8-bit products has enough range for the selected layer sizes.

B. PE Array

The PE array (Fig. 4) instantiates 16 PEs in a 4×4 grid. Each row receives one activation lane, and each column receives one

weight lane; the operands are broadcast along the rows and columns, and each PE accumulates its result locally (an output-stationary organization), so all 16 PEs operate in parallel and the array produces one 4×4 output tile after the accumulation loop finishes.

The a_bus contains four 8-bit activation values, while the b_bus contains four 8-bit weight values. The output c_bus is flattened from 16 accumulator outputs, each with 24 bits.

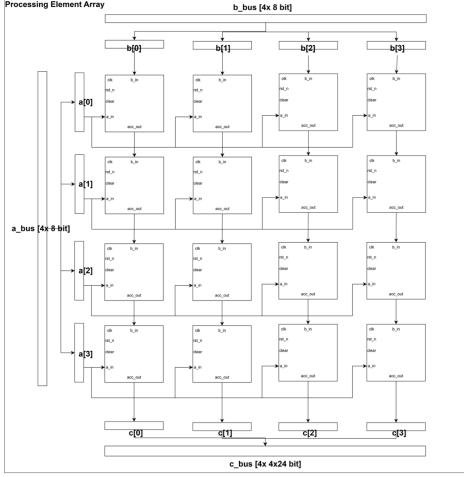


Fig. 4. Processing element array.

C. Controller FSM

The controller uses four states (Fig. 5). In IDLE, it waits for START. In CLEAR, it clears all PE accumulators for one cycle. In COMPUTE, it asserts the PE-array enable and increments the A and B buffer addresses until all K terms have been processed. In DONE, it raises DONE and waits for START to return low before going back to IDLE. A separate writeback sequencer then serializes the 16 accumulator values into the C buffer and raises a writeback-done flag.

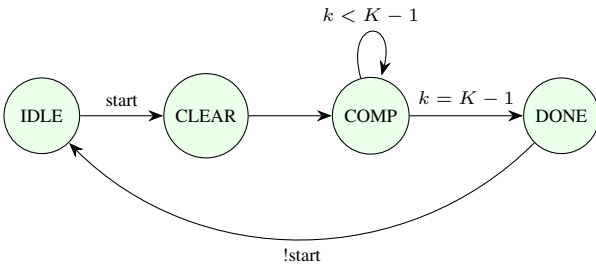


Fig. 5. Controller state machine.

D. SRAM Buffers

The input and output buffers are implemented with physical 256×8 single-port SRAM macros. The A buffer uses four macros, one for each row lane. The B buffer also uses four macros, one for each column lane. The C buffer stores 24-bit accumulator outputs by using three 8-bit SRAM macros as

byte planes of one 24-bit word. In total, the design uses eleven SRAM macros, 2,816 bytes of storage; the roughly 280 KB of INT8 generator weights are streamed through these tiles from outside rather than stored on chip.

The SRAM wrapper has a behavioral memory model for simulation and a blackbox branch for synthesis; the power pins are not tied to logic constants in RTL but connected by the physical power delivery network during place and route. The SRAM read has one cycle of latency, so the top level delays the PE enable and completion flags by one cycle to keep data and control aligned, which prevents the PE array from accumulating invalid data after a read request.

V. CHIP-LEVEL INTEGRATION

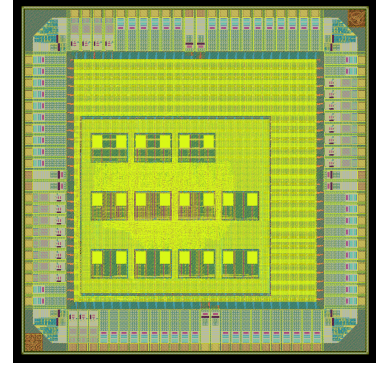


Fig. 6. Final chip layout ($2935 \times 2935 \mu\text{m}$ slot): the DLA macro with its 11 SRAMs, the padding, and the template's identification blocks.

The hardened accelerator exposes 53 signal bits, but the target shuttle slot provides only 20 general-purpose digital pads next to its 60 analog, 8 supply, clock, and reset pads. Instead of a pin-per-signal mapping, the chip core wraps the accelerator behind a synchronous 4-wire serial bridge (SCLK, MOSI, CS_N, MISO). The bridge treats SCLK as a data input: all three inputs are double-flop synchronized into the core clock domain and SCLK is edge-detected, deliberately avoiding a second clock domain. A frame is shifted MSB-first while CS_N is low as a 2-bit command plus a 10-bit address, write frames appending 8 data bits: WRITE_A and WRITE_B load the operand buffers, START launches one tile, and READ_C returns one 24-bit two's-complement result on MISO. Raising CS_N aborts the frame safely. Three status flags (busy, done, writeback-done) bypass the link and drive pads directly for oscilloscope-visible liveness at bring-up. The chip uses 9 of the slot's 91 pads; the remaining bidir pads are tied off.

The padding is the wafer.space GF180MCU project template [11] for a $2935 \times 2935 \mu\text{m}$ workshop slot. One consequence of the template is that the pad supply and the core supply are shorted into a single rail on every pad, so the chip is single-supply by construction. The adopted operating point is therefore 3.3 V for the whole die: the logic and SRAM are 3.3 V parts, and the foundry I/O cells are officially characterized for 3.3 V operation, so every cell on the die

operates in a foundry-characterized corner and the I/O levels are directly compatible with a 3.3 V microcontroller host. Fig. 6 shows the final chip layout.

VI. SOFTWARE TO HARDWARE FLOW

The system is separated into the hardware core and the verification environment. The hardware core is the DLA engine. It contains the A and B input buffers, the C output buffer, the controller, and the 4×4 PE array. This is the part that is hardened and taped out. The verification environment is written in Python and Verilog. It prepares quantized weights, input vectors, and reference outputs, then drives the accelerator through its normal read and write ports—or, at chip level, through the serial protocol exactly as an external host would. On the finished chip, the same role is played by a small microcontroller that streams the tile schedule over the serial link.

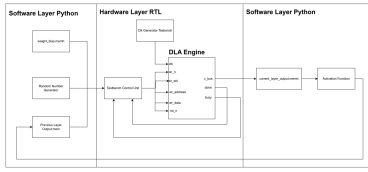


Fig. 7. Hardware and software flow.

As shown in Fig. 7, Python prepares the input data and memory contents, then the RTL testbench loads the data into the hardware engine. After the engine finishes, the output is read back and passed to the activation-function stage. This split keeps the accelerator simple, while still allowing the complete GAN pipeline to be checked.

A. Quantization and Requantization

The original model weights are stored as 32-bit floating-point values. Before they can be used by the integer array, every weight and bias tensor is converted to signed 8-bit values using one scale factor per tensor. The scale comes from the largest absolute value in the tensor, and all scales are saved in a manifest file so the testbench can reuse the same values.

The DLA only returns the integer dot product of one tile. The rest of the math is done around it in integer form, so the hardware result matches the Python reference. For the two hidden layers, the integer result is rescaled, the bias is added, and a ReLU activation is applied before the value is quantized again to 8 bits for the next layer. For the output layer, the result is turned into a fixed-point value, passed through a tanh lookup table, and then mapped to a grayscale pixel. The lookup table is used so that tanh does not have to be computed directly in hardware.

B. Why INT8

The operand width sets multiplier area, buffer depth, and weight-stream volume at once, so it was chosen by measurement. Each candidate format was applied to the same FP32 checkpoint under this project’s quantization scheme and the

TABLE II
NUMERIC FORMATS FOR THIS GENERATOR: ACCURACY AGAINST AN EXACT FP64 RENDER (10 LATENTS, 7,840 PIXELS), AND HARDWARE COST. PE AREA IS SYNTHESIZED ON THE 3.3 V CELLS; ACCUMULATOR WIDTH IS WHAT 256 ACCUMULATED PRODUCTS NEED.

Format	Pixel error (gray levels)		Weights (KiB)	PE area (μm^2)	Acc. (bits)
	mean	max			
INT4	22.26	255	138	6,245	15
INT8	1.22	72	276	15,963	23
INT16	0.01	1	552	47,037	39
FP16	0.06	5	552	—	—

rendered image compared against an exact FP64 render of the same latent, with error in gray levels of 255; the hardware side is the synthesized area of one PE at each width, the accumulator sized to what $K = 256$ requires (Table II).

INT4 is not viable here: a per-tensor scale leaves only seven magnitude steps, and accumulating that coarseness over 256 terms destroys the digit rather than degrading it. INT16 is essentially exact but costs $2.95\times$ the PE area, a 39-bit accumulator, and double the weight traffic—which, since weight streaming dominates end-to-end latency (Section IX), would double the time per image. FP16 is no more accurate than INT16 while needing the same memory and a different datapath (exponent alignment, normalization, rounding); no PE area is quoted because none was built. INT8 sits at the knee: 1.2 gray levels of average error is invisible in a 28×28 digit, the 16-bit product fits the 24-bit accumulator, and the operand width matches the SRAM macro’s native word, so the buffers need no packing logic.

VII. VERIFICATION

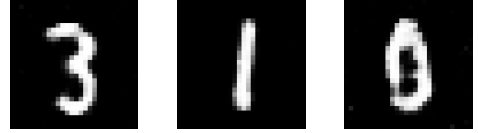


Fig. 8. Generated digit samples.

TABLE III
VERIFICATION RESULTS.

Testbench	Check	Result
d3004_tb	one GEMM, single lane	PASS
d3004n4_tb	full $N=4/K=256$ tile, all A/B macros	PASS
g3005_tb	one layer row	PASS
g300_pipeline_tb	full 784-pixel image	PASS
chip_core_dla_tb	pad-level serial protocol	PASS
gate-level (unit)	routed netlist, $N=4/K=256$ vectors	PASS
gate-level (image)	routed netlist, full 784-pixel image	PASS

To verify the GAN RTL result, the generator is mapped onto the DLA as a sequence of tiled dense layers. For each tile, four output neurons are computed at the same time: the four weight rows are written to the A buffer bank, and the

shared input vector is written into the B buffer. The DLA then performs the dot products and stores the results in the C buffer. Layer L0 uses the 64-dimensional latent vector as input, and layers L2 and L4 use the 256-dimensional activations of the previous layer; since the array has four output lanes, L0 and L2 each require 64 tiles, while L4 requires 196 tiles. The same datapath is reused for all layers.

Table III summarizes the stack; all testbenches check against Python golden vectors. The two gate-level runs re-simulate the routed netlist itself—inserted buffers and roughly 7,700 antenna diodes included—and reproduce the golden results bit-exactly, closing the gap between “the RTL is correct” and “the taped-out netlist is correct.” The stack caught two real integration bugs: the A and B buffers use different write-address layouts (row-major versus k -major), and the serial READ_C path needed an extra wait state because a C-buffer address update is not visible to the registered SRAM read until the next cycle.

Fig. 8 shows examples of generated 28×28 images. The visual quality is limited by the small generator and by INT8 quantization, but the images are enough to verify that the full software-to-hardware path is running.

VIII. PHYSICAL IMPLEMENTATION

The design is implemented with LibreLane [12] on the GF180MCU open PDK [13] in two stages: the accelerator is hardened as a macro, and the macro GDS/LEF/liberty views are then consumed by the padding integration.

A. Single-Supply 3.3 V Library Migration

The buffer SRAM macro [14] is a 3.3 V IP, but the open PDK ships only 5 V standard cells, so a first version mixed a 5 V core with 3.3 V SRAMs—workable for layout checks but needing level shifting on silicon. The design was migrated to the third-party `gf180mcu_as_sc_mcu7t3v3` native-3.3 V library, which ships nine liberty corners and a LibreLane integration, making the die single-supply. Bringing it up surfaced three real bugs in its flow integration (a missing timing-corner variable, a synthesis-exclusion filename mismatch, and non-synthesizable `$random` initializers in six sequential-cell models that crashed lint), all fixed at the library source. The 3.3 V cells are much faster than the 5 V ones: the first run at the old 75 ns clock closed with +44 ns of setup slack, so the clock was retimed to 40 ns (25 MHz).

B. Hardening the Accelerator Macro

TABLE IV
ANTENNA CLOSURE SWEEP (PLACEMENT DENSITY, $K=256$ NETLIST, 40 NS CLOCK). ALL RUNS ARE DRC-, LVS-, AND TIMING-CLEAN.

Density (%)	Antenna nets	Setup slack (ns)	Result
61	2	+15.53	worse
60	5	+15.75	
62	1	+15.60	
63	0	+15.12	clean sign-off identical DEF
63 (re-run)	0	+15.12	

A few configuration points were essential. A synthesis define keeps the SRAM as a hard macro—without it, the memory is silently built from flip-flops. The eleven macros are registered with their LEF/GDS/liberty views and placed explicitly on a 4×3 grid with routing channels. The SRAM power pins only reach Metal3, so the macro power-grid connection had to be made on Metal3 instead of the default higher layers. Routing was allowed up to Metal5, standard-cell padding was added for diode pin access, and antenna diodes are inserted heuristically at placement time with the in-router antenna repair disabled.

The antenna check required an empirical closure method. In-router antenna repair aborts in this flow because its verification reroute fails pin access on already-placed clock-net diodes, and the residual violation was a wire-length problem on a long buffered net already carrying several diodes, so more diodes could not help either. Larger floorplan changes (clustering, row swaps, fanout caps) consistently made the count worse by creating new long routes. What worked—for the earlier 5 V version and again here—is a small deterministic perturbation of placement density on the same floorplan, which re-rolls which nets route long. Table IV shows the sweep: density 63 reaches zero, and re-running it reproduces a byte-identical routing database, so the result is deterministic rather than a routing fluke.

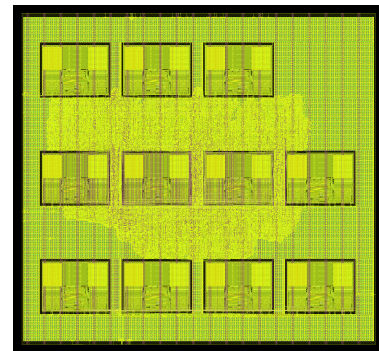


Fig. 9. Hardened accelerator macro ($1600 \times 1500 \mu\text{m}$) with the eleven SRAM macros on a 4×3 grid.

Fig. 9 shows the resulting macro layout and Table VI the sign-off. Magic reports zero DRC errors, Netgen reports zero LVS errors, and the antenna report has zero violating nets and pins. The design meets the 40 ns clock at all nine STA corners: the worst setup slacks per corner family are +23.41 ns (TT), +15.12 ns (SS, 125 °C, 3.0 V), and +26.54 ns (FF), and the worst hold slacks are +0.260, +0.333, and +0.150 ns respectively. The reported nominal power is 162 mW, almost entirely internal power estimated with the flow’s default switching activity, and should be read as a conservative tool estimate rather than a measurement.

C. Area and Timing Detail

The lower panel of Table VI resolves the core area. The eleven SRAM macros take 0.745 mm^2 , twice the 0.384 mm^2 of all placed standard cells, so the floorplan is memory-dominated

TABLE V
WORST SLACK PER PVT CORNER AT THE 40 NS CLOCK (WORST OF THE THREE INTERCONNECT MODELS AT EACH POINT). ZERO SETUP AND ZERO HOLD VIOLATIONS EVERYWHERE.

Corner	Macro (Stage 1)		Chip (Stage 2)	
	setup	hold	setup	hold
SS, 125 °C, 3.0 V	+15.12	+0.333	+21.67	+0.345
TT, 25 °C, 3.3 V	+23.41	+0.260	+25.36	+0.483
FF, −40 °C, 3.6 V	+26.54	+0.150	+27.14	+0.329
Clock skew (worst)	0.65 ns		1.74 ns	
Critical path	24.88 ns		macro-limited	

TABLE VI
HARDENED MACRO SIGN-OFF AND CORE AREA BREAKDOWN.

Metric	Value
Process	GF180MCU, 180 nm
Standard cells	gfl180mcu_as_sc_mcu7t3v3 (3.3 V)
Die area	1600 × 1500 μm^2
Core area (utilization)	2.326 mm ² (48.6%)
Clock period	40 ns (25 MHz)
Worst setup slack (9 corners)	+15.12 ns
Worst hold slack (9 corners)	+0.150 ns
Magic DRC / Netgen LVS	0 / 0 errors
Antenna	0 nets, 0 pins
Power (nominal, tool estimate)	162 mW
Routed wirelength	867 mm
<i>Core area breakdown (cells, area)</i>	
SRAM macros (11 × 256 × 8)	11; 745,485 μm^2 (32.1%)
Combinational logic	9,864; 238,528 μm^2 (10.3%)
Well taps	11,540; 50,665 μm^2 (2.2%)
Antenna diodes	7,763; 34,083 μm^2 (1.5%)
Sequential cells	406; 32,976 μm^2 (1.4%)
Buffers, inverters, ties	2,138; 28,128 μm^2 (1.2%)
Fill and decap	61,450; 1,053,520 μm^2 (45.3%)

even though the buffers hold only 2,816 bytes: a 256×8 macro costs 67,771 μm^2 however little of its depth a layer uses. Within the standard cells the arithmetic is a minority — 0.239 mm² of combinational logic against 0.085 mm² of well taps and the 7,763 antenna diodes that the closure strategy of Section VIII inserts at placement. The rest of the core is decap and fill.

Timing is signed off at nine corners — three PVT points crossed with minimum, nominal, and maximum interconnect — and Table V gives the worst interconnect model at each PVT point. Setup is governed by the slow corner and hold by the fast one, both with margin. The 40 ns period is conservative by design: +15.12 ns of setup slack puts the true critical path at 24.88 ns, so the macro would close near 40 MHz and the 25 MHz operating point buys a 1.6× guard band on a first tapeout in an unfamiliar library. That path runs from an SRAM read through the operand multiplexing into the MAC and back to the accumulator register — making A and B SRAM rather than registers is what puts a macro read on a combinational path, and at the slow corner the read alone is some 13 ns of it. Hold slack is the thinnest number in the sign-off at +0.150 ns and is set by clock-tree skew (0.65 ns) rather than by the

period: it was +0.117 ns at the earlier 75 ns clock. It is what to watch if a floorplan change moves the clock tree.

D. Full-Chip Implementation

TABLE VII
FULL-CHIP SIGN-OFF RESULTS.

Metric	Value
Slot die	2935 × 2935 μm (8.614 mm ²)
Core area	4.195 mm ² (72.0% utilized)
Pads	91 (9 used), 3.042 mm ²
Macro area (DLA, ID, logo)	2.441 mm ²
Chip-level standard cells	13,761 (0.580 mm ²)
Instances / macros	71,668 / 3
Magic DRC / KLayout DRC	0 / 0
Chip-level LVS	0 errors
Antenna	0 nets, 0 pins
GDS XOR (Magic vs. KLayout)	0
Worst setup slack (9 corners)	+21.67 ns
Worst hold slack (9 corners)	+0.329 ns
Worst IR drop	0.31 mV

The padding integration re-runs the complete flow with the hardened macro as a black box. Two macro-interface issues had to be resolved: the macro’s behavioral SRAM wrapper had to be black-boxed for the chip-level static timing reader, and four short power-strap fragments that the macro LEF exports as pins lie entirely under a Metal5 routing obstruction, which the chip-level power-grid builder rejects; they were reclassified as obstructions, which is electrically equivalent and strictly more conservative.

Chip-level timing needed particular care. The first sign-off run reported infinite slack at every corner—a hollow result: with no pad liberty loaded, the timing tool treated every pad as an empty black box, the clock defined on the clock pad’s output degraded to a virtual clock, and nothing was constrained. Loading the foundry pad liberty at its 3.3 V corners fixed it, and the now-real numbers exposed one more constraint bug: the template’s blanket input delays imposed hold checks on the three synchronizer-guarded asynchronous serial inputs, which were replaced by false paths. After both fixes all nine corners close on real paths (Table VII).

IX. PERFORMANCE

A. Latency Breakdown

TABLE VIII
END-TO-END LATENCY FOR ONE 28×28 IMAGE, AT A 25 MHz CORE CLOCK AND THE NOMINAL SCLK OF 3.125 MHz (CLK/8). CYCLE COUNTS ARE MEASURED FROM THE FULL-IMAGE SIMULATION.

Stage	Frames	Time (ms)	Share
Weight tiles $\rightarrow$ A buffer	331,776	2,123.4	98.85%
Input vector $\rightarrow$ B buffer	768	4.9	0.23%
START commands	324	1.2	0.06%
Compute + writeback (on chip)	324 tiles	3.5	0.16%
Result readback from C	1,296	14.9	0.70%
Total		2,148	0.47 img/s

The array performs 16 MAC operations per cycle, a peak of 0.8 GOPS at 25 MHz; quoting only that would be misleading, so this section separates what the array costs from what feeding it costs. The on-chip cycle counts are measured by instrumenting the full-image RTL simulation per controller state, and the link times follow from the frame lengths of Section V.

Per tile the engine is busy 273 cycles: one CLEAR, 256 accumulation cycles, and 16 writeback cycles serializing the accumulators into C. Over 324 tiles that is 88,452 cycles, or 3.54 ms at 40 ns — the “pure compute” figure, and 20% of peak. The shortfall is structural, not a scheduling defect: the generator evaluates a matrix–vector product, so only the four PEs of B’s first column produce useful neurons while the other twelve recompute against unused columns; batching four latents would recover most of it with no hardware change. Feeding the array costs far more. Each tile needs its four weight rows written across the full $K = 256$ depth, 1,024 bytes, and every byte is one 20-bit frame. The bridge synchronizes and edge-detects SCLK, so the link is specified at $SCLK \leq clk/8$ — 3.125 MHz, 320 ns per bit, 6.4 μ s per byte (Table VIII).

End-to-end latency is therefore **2.15 s per image** (0.47 images/s) against 3.5 ms of arithmetic: the serial link is 99.8% of it and weight streaming alone 98.9%, giving an effective 263 kOPS. The bottleneck is not link width alone but the workload’s zero weight reuse — a matrix–vector product touches every weight once, so 1.01 bytes must cross the interface per MAC however that interface is built. Even an idealized parallel port taking one byte per core clock leaves weight loading at 331,776 of 426,647 cycles, still 78%. Some traffic is avoidable: the schedule writes the full 256-deep A tile even for L0, whose inputs are 64 wide, so a host that pre-zeroes the tail sends 282,624 bytes and finishes in 1.83 s. This is the trade the pad budget forced — 20 pads and 2.8 KB of SRAM against 280 KB of weights — and it is where future work belongs: a burst write that sends the address once would remove 12 of every 20 bits, and a wider host bus scales further.

TABLE IX
COMPARISON WITH REPRESENTATIVE DEEP LEARNING ACCELERATORS. THROUGHPUT IS PEAK ($2 \times \text{MACs} \times \text{FREQUENCY}$); EFFICIENCY IS PEAK THROUGHPUT OVER REPORTED POWER. OUR POWER IS A TOOL ESTIMATE, NOT A MEASUREMENT.

Design	Tech. (nm)	Area (mm ²)	Peak (GOPS)	Eff. (GOPS/W)	Open flow
DianNao [3]	65	3.02	452	932	No
Eyeriss [4]	65	12.25	67.2	242	No
Wang <i>et al.</i> [7]	40	11.97	54.61	567	No
OpenSpike [5]	130	33.3	—	56.8	Yes
This work	180	2.33	0.8	4.9	Yes

B. Comparison

Table IX places this work against three fabricated accelerators on commercial nodes [3], [4], [7] and one built, like ours, entirely with open tools on an open PDK [5]. Throughput and efficiency are peak figures ($2 \times \text{MACs} \times \text{frequency}$) so that very different scales compare on one basis.

The gap should be read plainly: two to three orders of magnitude below the commercial-node parts in peak throughput and about two in efficiency. Almost none of it is architectural. A 4×4 array at 25 MHz has $31 \times$ fewer multipliers than Eyeriss at $8 \times$ the clock, which accounts for the throughput difference by itself, and the efficiency difference is what 180 nm at 3.3 V costs against 65 nm and below. The comparison that isolates the contribution is OpenSpike, the other open-PDK, open-flow tapeout: at a similar scale and clock this design occupies 2.33 mm² against 33.3 mm², uses foundry-supplied hard SRAM macros rather than generated memory, and is carried through padding integration to a signed-off chip. What this work offers is not performance but reproducibility — every step from the quantization scripts to the final GDS runs on public tools and a public PDK.

X. DISCUSSION

The performance results of Section IX say something more general about this class of design than about this chip. A tile engine with no weight reuse is interface-bound at any interface width, and the 2.8 KB of on-chip SRAM against 280 KB of weights was not an oversight but the pad budget’s consequence. The three levers that would matter are, in order: batching several latents through the array’s unused columns, which costs no hardware and recovers most of the $5 \times$ utilization gap; a burst write command that amortizes the address over many bytes; and a wider host bus. None of them requires the datapath to change.

The implementation also shows a trade-off between timing and antenna closure: floorplans that shortened the SRAM-to-MAC critical path consistently increased the antenna count, and the priority here was a clean physical sign-off rather than the fastest possible clock. The earlier voltage-domain concern was resolved structurally—the padding shorts the pad and core supplies, so the single-supply 3.3 V operating point removes the level-shifting problem entirely, at the cost of

3.3 V (rather than 5 V) I/O levels, which matches the intended microcontroller host anyway. A true dual-rail variant was evaluated and deferred because the only available dual-voltage pad library is not yet silicon-ready. A bring-up plan for the returned silicon is prepared: current-limited power-on, status-pin checks, replay of the unit-test golden vectors over the serial link from a bit-banging 3.3 V microcontroller, and finally a full streamed digit render.

XI. CONCLUSION

This work implements a small INT8 GAN-generator accelerator on GF180MCU and carries it to a tapeout-ready chip with a fully open-source flow. The design replaces behavioral memory arrays with eleven SRAM macros, uses a 4×4 PE array with a native $K=256$ accumulation depth for tiled matrix multiplication, and verifies the output against Python references at every level up to a bit-exact full-image render on the routed post-layout netlist. Both the hardened macro and the integrated chip pass DRC, LVS, and antenna checks cleanly and close timing at 40 ns across nine corners on a single 3.3 V supply. The main remaining work is silicon bring-up and improving data reuse for higher throughput.

ACKNOWLEDGMENT

The authors thank the open-source EDA community, the GlobalFoundries open PDK program for GF180MCU and the SRAM macro, and the wafer.space chipathon program for the padding template and the shuttle slot targeted by this work.

REFERENCES

- [1] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proc. ISCA*, 2017.
- [2] C. Silvano *et al.*, “A survey on deep learning hardware accelerators for heterogeneous HPC platforms,” *ACM Computing Surveys*, 2025.
- [3] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “DianNao: a small-footprint high-throughput accelerator for ubiquitous machine-learning,” in *Proc. ASPLOS*, 2014, pp. 269–284.
- [4] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: an energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [5] F. Modaresi, M. Guthaus, and J. K. Eshraghian, “OpenSpike: an Open-RAM SNN accelerator,” in *Proc. IEEE ISCAS*, 2023.
- [6] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF CVPR*, 2018.
- [7] C. C. Wang, S. H. Luo, H. C. Wu, R. G. B. Sangalang, C. P. Jou, and H. Hsia, “A 54.61 GOPS 96.35 mW digital logic accelerator for underwater object recognition DNN using 40 nm CMOS process,” in *Proc. IEEE 6th Int. Conf. AI Circuits and Systems (AICAS)*, 2024.
- [8] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Proc. NeurIPS*, 2014.
- [9] C. Singh, “GAN/VAE pretrained PyTorch models,” <https://github.com/csinva/gan-vae-pretrained-pytorch>.
- [10] “DLA Engine,” GitHub, https://github.com/wlmoi/DLA_Engine.
- [11] wafer.space, “GF180MCU project template,” <https://github.com/wafer-space/gf180mcu-project-template>.
- [12] The LibreLane project, “LibreLane: an open-source RTL-to-GDSII flow,” <https://github.com/librelane/librelane>.
- [13] GlobalFoundries and Google, “GF180MCU open-source PDK,” <https://github.com/google/gf180mcu-pdk>.
- [14] R. T. Edwards, “GF180MCU OCD IP SRAM macros,” https://github.com/RTimothyEdwards/gf180mcu_ocd_ip_sram.